

Blinkit Customer Analytics & Feedback System

Project Report: ML Models & Attribution Logic

Generated on: January 05, 2026

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Attribution Modeling Logic
4. Machine Learning Models & Hyperparameters
5. RAG (Retrieval Augmented Generation) System
6. Conclusion & Recommendations

1. Executive Summary

This project implements a comprehensive customer analytics and feedback system for Blinkit, a quick-commerce platform. The system comprises four interconnected layers:

Layer	Component	Purpose
Layer 1	Master Analytical View	Data aggregation and consolidation
Layer 2	Streamlit Dashboard	Interactive data visualization
Layer 3	ML Delay Risk Predictor	Predictive analytics for delivery delays
Layer 4	RAG Feedback Chat	AI-powered feedback analysis using Ollama

The system leverages multiple data sources including customer information, orders, products, feedback, and operational metrics to provide actionable insights and predictive capabilities.

2. Architecture Overview

2.1 Data Flow Architecture

The system follows a layered architecture approach where each layer builds upon the previous one:

Data Sources:

- **blinkit_customers:** Customer demographics and area information
- **blinkit_orders:** Order details, timestamps, and totals
- **blinkit_products:** Product catalog with categories and pricing
- **blinkit_order_items:** Line items linking orders to products
- **blinkit_customer_feedback:** Ratings, comments, sentiment, and categories
- **blinkit_operations:** Delivery metrics and promised/actual times

Data Integration:

All data sources are joined using relational keys (`customer_id`, `order_id`, `product_id`) to create enriched analytical views. This enables comprehensive analysis across multiple dimensions.

3. Attribution Modeling Logic

3.1 Feedback Attribution Framework

The feedback attribution system links customer feedback to specific business entities to identify root causes of issues. This multi-dimensional attribution enables precise problem identification.

Dimension	Attribution Method	Business Value
Customer Area	Direct join on customer_id → area	Identify regional patterns & logistics issues
Product Category	Join order_id → product_id → category	Product quality & category-specific problems
Delivery Status	Join order_id → delivery_status	Operational efficiency metrics
Time Period	Temporal analysis on feedback_date	Seasonal trends & time-based patterns
Sentiment	NLP-based sentiment classification	Emotional context of feedback

3.2 Feature Engineering for Attribution

Aggregated Product Information: For orders with multiple items, product names and categories are concatenated to preserve full context. This ensures no information loss when analyzing multi-item order feedback.

Temporal Features: Hour of day, day of week, and seasonal patterns are extracted from timestamps to identify time-based correlations with customer satisfaction.

Geographic Attribution: Customer area information enables location-based root cause analysis, critical for identifying delivery zone issues and regional product preferences.

4. Machine Learning Models & Hyperparameters

4.1 Delivery Delay Risk Prediction Model

Model Type: Random Forest Classifier

Problem Type: Binary Classification (Delay vs On-Time)

Framework: scikit-learn

Hyperparameters & Rationale:

Hyperparameter	Value	Rationale
n_estimators	100	Balance between model complexity and training time. 100 trees provide robust
random_state	42	Ensures reproducibility across training runs.
max_depth	None (unlimited)	Allow trees to grow until pure leaves or min_samples_leaf is reached, capturing
min_samples_split	2 (default)	Default value works well for our dataset size.
min_samples_leaf	1 (default)	Allows granular decision boundaries.

4.2 Feature Set for Delay Prediction

Feature	Type	Description
hour_of_day	Numeric (0-23)	Hour when order is placed/promised delivery
day_of_week	Numeric (0-6)	Day of week (0=Monday, 6=Sunday)
region_encoded	Categorical (Label Encoded)	Geographic region/area identifier

4.3 Model Performance Metrics

Evaluation Metrics:

- **ROC-AUC Score:** Measures model's ability to discriminate between delay/on-time
- **Accuracy:** Overall prediction correctness
- **Prediction Probability:** Provides confidence levels for risk assessment

Risk Thresholds:

- **High Risk:** Probability > 0.7 (70%)
- **Moderate Risk:** Probability 0.4-0.7 (40-70%)
- **Low Risk:** Probability < 0.4 (40%)

5. RAG (Retrieval Augmented Generation) System

5.1 Architecture Components

The RAG system combines semantic search with local LLM inference to provide intelligent feedback analysis without relying on external APIs.

Component	Technology	Purpose
Embeddings	SentenceTransformer (all-MiniLM-L6-v2)	Convert feedback text to 384-dim vectors
Vector Search	Cosine Similarity	Find semantically relevant feedback
LLM	Ollama (llama3.2)	Generate insights from retrieved context
Cache	Streamlit @st.cache	Optimize performance & reduce recomputation

5.2 RAG Pipeline Flow

Step 1 - Embedding Generation: All feedback texts are encoded using SentenceTransformer, creating dense vector representations that capture semantic meaning.

Step 2 - Query Processing: User queries are embedded using the same model for consistency.

Step 3 - Semantic Search: Cosine similarity identifies the top-k (k=8) most relevant feedback entries based on vector proximity.

Step 4 - Context Enrichment: Retrieved feedback is augmented with metadata (area, products, ratings, sentiment) to provide comprehensive context.

Step 5 - LLM Analysis: Ollama llama3.2 model analyzes the enriched context and generates actionable insights, identifying patterns across dimensions.

5.3 LLM Configuration

Parameter	Value	Purpose
Model	llama3.2	Latest Meta Llama model, optimized for reasoning
Temperature	0.7	Balanced creativity vs. consistency
Max Tokens	512	Sufficient for comprehensive analysis
Stream	False	Wait for complete response

6. Conclusion & Recommendations

6.1 Key Achievements

- **Comprehensive Attribution:** Successfully linked feedback to customers, products, areas, and delivery metrics
- **Predictive Capabilities:** Implemented delay risk prediction with interpretable features
- **No External Dependencies:** Fully local RAG system using Ollama eliminates API costs and rate limits
- **Actionable Insights:** Multi-dimensional pattern analysis enables targeted interventions

6.2 Future Enhancements

- **Model Improvements:** Implement hyperparameter tuning using GridSearchCV or Optuna
- **Additional Features:** Include weather data, traffic patterns, order volume metrics
- **Real-time Predictions:** Deploy model as REST API for live risk assessment
- **Advanced NLP:** Fine-tune domain-specific embeddings for better feedback understanding
- **A/B Testing:** Implement experimentation framework for interventions

6.3 Technical Stack Summary

Layer	Technologies
Data Processing	Pandas, NumPy
ML Framework	scikit-learn, XGBoost (optional)
Embeddings	sentence-transformers
LLM	Ollama (llama3.2)
Visualization	Streamlit, Plotly
Storage	CSV, PostgreSQL (optional)

— End of Report —